



University of
BRISTOL

University of Bristol
Faculty of Science and Engineering

Fundamentals of System Security

Individual Coursework Report

Student Name: Yi Li
Student ID: 2829967
Unit code: COMSM0122
Coursework: Tiered Infrastructure Security

Bristol, UK
November 2025

Contents

1	Task 1: Infrastructure Design Patterns	2
1.1	(a) Software Design Pattern	2
1.2	(b) Separation of Concerns	2
2	Task 2: Infrastructure Health	3
2.1	(a) HAProxy Jinja2 Templates and Playbook	3
2.2	(b) Traffic Simulation	3
2.2.1	Roundrobin Algorithm	4
2.2.2	Leastconn Algorithm	4
2.2.3	Source Algorithm	5
2.2.4	Summary of Load Balancing Efficiency	5
2.3	(c) Resource Scaling	6
2.3.1	Memory Test	7
2.3.2	Disk Test	8
2.3.3	CPU Test	10
2.4	(d) VM Replication	12
3	Task 3: Access Control Matrix	14
3.1	(a) Access Control Matrix	14
3.2	(b) Bell-LaPadula Confidentiality	14
3.2.1	(1) Simple security property (no read-up)	14
3.2.2	(2) Star property (no write-down)	15
3.2.3	(3) Enforcement (Bell-LaPadula)	15
3.3	(c) Biba Integrity	16
3.3.1	(1) Simple Integrity Property (no read-down)	16
3.3.2	(2) Star (Property (no write-up)	16
3.3.3	(3) Enforcement of Biba integrity	16
3.4	(d) Conflict Identification and Resolution	17

1 Task 1: Infrastructure Design Patterns

1.1 (a) Software Design Pattern

This architecture most closely resembles multi-tier architecture (Factory model) because Each component of the system has a specific task. Terraform creates VMS all at once on Proxmox, while Ansible installs the corresponding software and configurations for each machine. HAProxy diverts user requests to different web machines. Web-01 and Web-02 run the Node.js API, app-01 acts as the business layer to handle the back-end logic, and admin-01 is responsible for management and monitoring. This approach offers better scalability. If the access volume increases, we only need to add a few more web machines (clone Web-03 and Web-04) in Terraform. In addition, the system also has excellent maintainability. If Node.js needs to be updated, we only need to update the web layer (Ansible role). Moreover, if HAProxy is configured incorrectly, it will only affect load balancing.

1.2 (b) Separation of Concerns

In this structure, the division of labor between Terraform and Ansible reflects the principle of separation of concerns. Terraform is responsible for creating virtual machines, networks and templates on Proxmox. It only cares about the infrastructure itself. After the virtual machine is created, Ansible installs HAProxy, applications, user configurations and system parameters. It only cares about the software and the operating environment. Therefore, when we modify Terraform, we do not change the application configuration. When we change the file configuration, we do not change the virtual machine itself, such as deleting or adding VMS. In terms of team cooperation, the project has a clear division of labor. People who is responsible for the virtual machine (Terraform) and people who is responsible configuration (Ansible) focus on their parts. Thus, the system is easier to manage.

2 Task 2: Infrastructure Health

2.1 (a) HAProxy Jinja2 Templates and Playbook

```
# roles/haproxy/tasks/main.yml
- name: Deploy HAProxy config (selectable algorithm)
  template:
    src: "haproxy_{{ haproxy_lb_alg }}.cfg.j2"
    dest: /etc/haproxy/haproxy.cfg
  notify: restart haproxy
```

Figure 1: Code for selecting the appropriate algorithm

In this task, Jinja2 templates and Ansible are used to define multiple haproxy.cfg to implement three different load balancing strategies (roundrobin, leastconn, and source).

Figure 1 shows the system dynamically selects one of the three templates based on the input variable haproxy_lb_alg, e.g. roundrobin, leastconn, or source.

```
# roles/haproxy/templates/haproxy_roundrobin.cfg.j2
backend http_back
    balance roundrobin
    option httpchk GET /api
{% for host in groups['web'] %}
    server {{ host }} {{ hostvars[host].ansible_host }}:3000
    check
{% endfor %}
```

Figure 2: Example HAProxy backend template (roundrobin)

Figure 2 shows an example of an HAProxy backend configuration template that uses the roundrobin algorithm. haproxy_roundrobin.cfg.j2 defines the backend service configuration of HAProxy, and balance roundrobin specifies the use of polling load balancing policies. The other two algorithms (leastconn and source) use the same configuration structure as Figure 1, except for the algorithm name specified in the balance instruction.

```
ansible-playbook -i inventories/dev/hosts.yml playbooks/site.yml
-e "haproxy_lb_alg=roundrobin"
```

Figure 3: Example of algorithm selection (roundrobin)

Figure 3 shows the command to switch the algorithm by changing the variable of haproxy_lb_alg = roundrobin.

2.2 (b) Traffic Simulation

In this task, experiments under increasing load (1,000–15,000 requests) were conducted to evaluate backend load allocation and server health across the three routing algorithms. And all the configurations are set by the code below:

```

# modules/vms/clone.yml
memory {
  dedicated = 1024
}
cpu {
  cores = 1
}
# modules/vms/template.yml
disk {
  datastore_id = var.datastore_id
  file_id      = proxmox_virtual_environment_download_file.
    ubuntu_cloud_image.id
  interface    = "virtio0"
  iothread     = true
  discard      = "on"
  size         = 10
}

```

Figure 4: Code for setting configs

2.2.1 Roundrobin Algorithm

As shown in Figure 1, web01 and web02 distribute requests evenly because Roundrobin is the algorithm that distributes requests in turns. The first request is for web-01, the second for web-02, the third for web-01 again, and so on in a cycle.

Table 1: Roundrobin algorithm results

Request	Web-01	Web-02	Health Status
1000	500	500	up
2000	1000	1000	up
3000	1500	1500	up
4000	2000	2000	up
5000	2500	2500	up
6000	3000	3000	up
7000	3500	3500	up
8000	4000	4000	up
9000	4500	4500	up
10000	5000	5000	up
11000	5500	5500	up
12000	6000	6000	up
13000	6500	6500	up
14000	7000	7000	up
15000	7500	7500	up

2.2.2 Leastconn Algorithm

From Figure 2, it can be seen that the Leastconn algorithm adjusts request distribution in real time based on the current load.

Table 2: Leastconn algorithm results

Request	Web-01	Web-02	Health Status
1000	475	525	up
2000	962	1038	up
3000	1435	1565	up
4000	1941	2059	up
5000	2492	2508	up
6000	2998	3002	up
7000	3470	3530	up
8000	3988	4012	up
9000	4531	4469	up
10000	5040	4960	up
11000	5540	5460	up
12000	6055	5945	up
13000	6537	6463	up
14000	7026	6974	up
15000	7528	7472	up

2.2.3 Source Algorithm

From Figure 3, it can be seen that the Source algorithm always forwards requests to the same backend server based on the client IP.

Table 3: Source algorithm results

Request	Web-01	Web-02	Health Status
1000	0	1000	up
2000	0	2000	up
3000	0	3000	up
4000	0	4000	up
5000	0	5000	up
6000	0	6000	up
7000	0	7000	up
8000	0	8000	up
9000	0	9000	up
10000	0	10000	up
11000	0	11000	up
12000	0	12000	up
13000	0	13000	up
14000	0	14000	up
15000	0	15000	up

2.2.4 Summary of Load Balancing Efficiency

Table 4: Backend request distribution based on HAProxy statistics

Algorithm	Web-01 (%)	Web-02 (%)	Balance Quality	Conclusion
Source	100	0	Poor	All requests directed to one node
Roundrobin	≈ 50	≈ 50	Acceptable	Occasional imbalance at high load
Leastconn	≈ 50	≈ 50	Excellent	Best request allocation

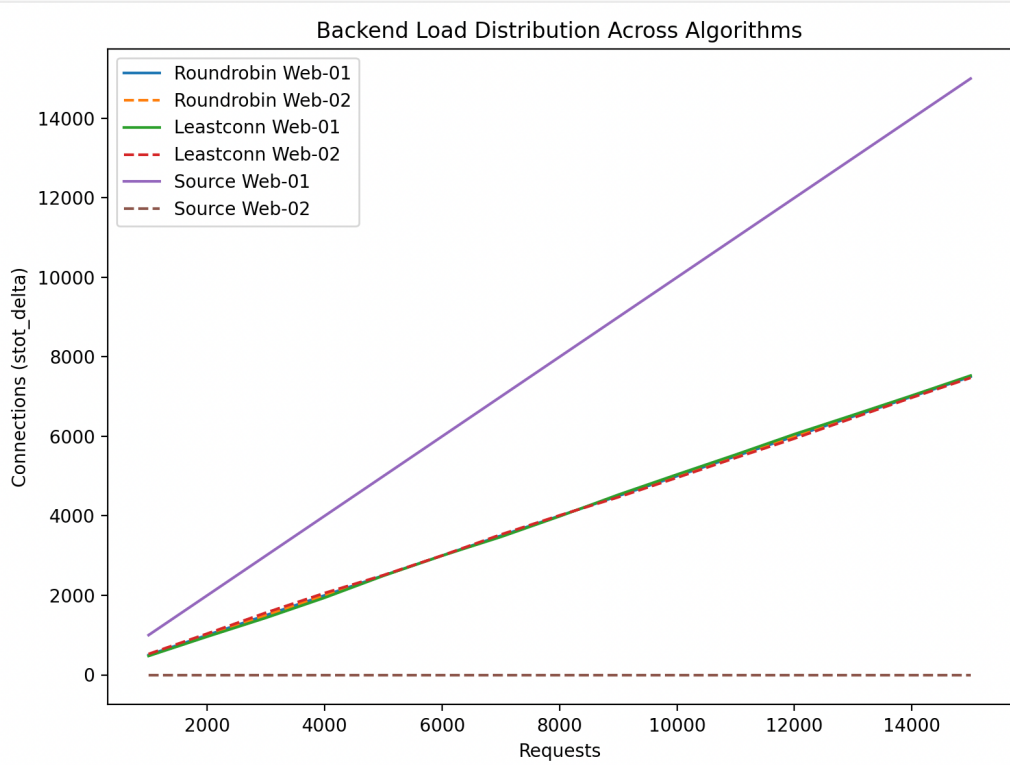


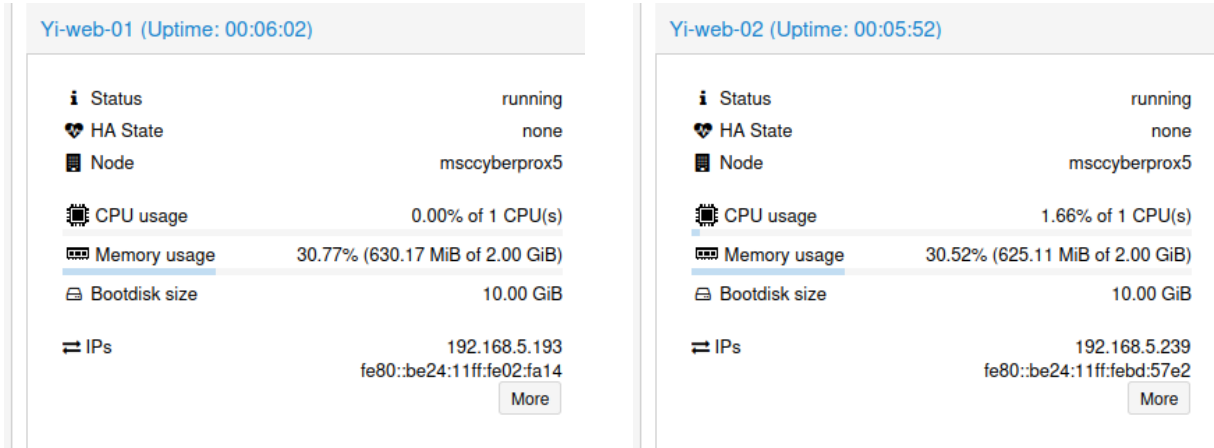
Figure 5: Backend load distribution across different HAProxy routing algorithms

Therefore, through the comparison of the three algorithms, Leastconn has the most balanced load and the best performance. Leastconn delivers the most efficient and stable load distribution, followed by Roundrobin, while Source demonstrates highly unbalanced behaviour.

2.3 (c) Resource Scaling

In this task, due to the huge amount of data collected caused by the three algorithms, the highest configuration operation results of each algorithm and the web configuration are provided in each section.

2.3.1 Memory Test



(a) web 01 with memory 2048MB

(b) web 02 with memory 2048MB

Figure 6: Configuration of two webs based on 2048MB memory

Table 5: Backend request distribution for different HAProxy algorithms - 2048MB

Request	Roundrobin		Leastconn		Source	
	Web-01	Web-02	Web-01	Web-02	Web-01	Web-02
1000	500	500	491	509	0	1000
2000	1000	1000	1013	987	0	2000
3000	1500	1500	1501	1499	0	3000
4000	2000	2000	1985	2015	0	4000
5000	2500	2500	2538	2462	0	5000
6000	3000	3000	3038	2962	0	6000
7000	3500	3500	3507	3493	0	7000
8000	4000	4000	3994	4006	0	8000
9000	4500	4500	4493	4507	0	9000
10000	5000	5000	4976	5024	0	10000
11000	5500	5500	5472	5528	0	11000
12000	6000	6000	5959	6041	0	12000
13000	6500	6500	6446	6554	0	13000
14000	7000	7000	6949	7051	0	14000
15000	7500	7500	7450	7550	0	15000

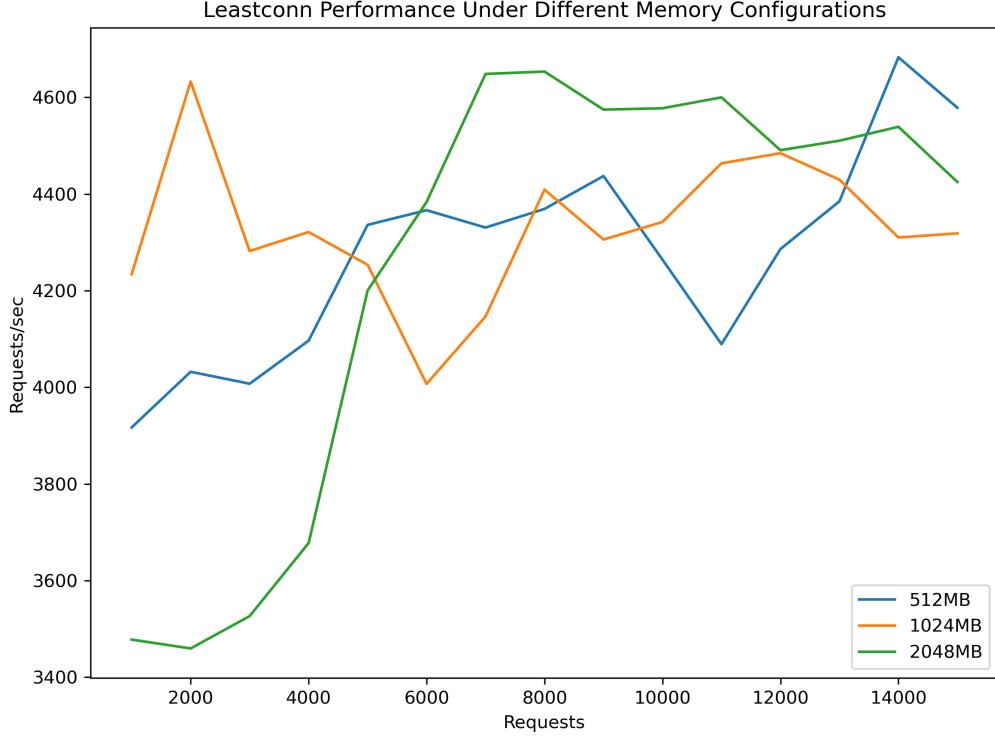
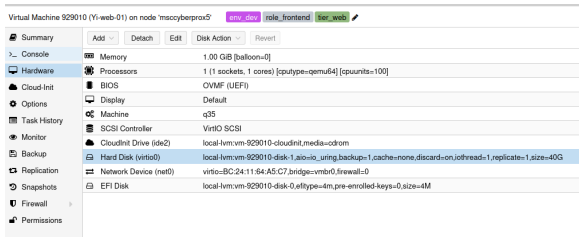


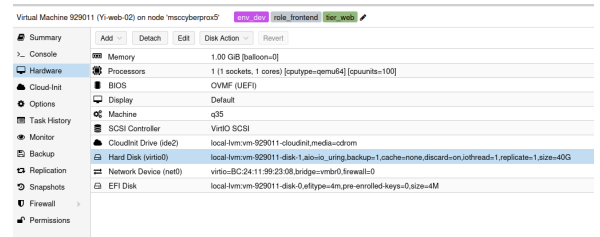
Figure 7: Leastconn Performance Under Different Memory Configs

Figure 6 and Table 5 provide an example of result with 2048MB memory and fixed CPU and hard disk. Roundrobin and Source have the same output and Leastconn makes a little bit changes. According to Figure 7, at low load levels (1,000–2,000 requests), the 2048MB configuration shows unexpectedly lower throughput, since the tests are likely very short and sensitive to background activity. For larger workloads, the 2048MB curve rises and stays close to or slightly above the 1024MB configuration, which confirms that additional memory brings only limited improvement beyond 1GB.

2.3.2 Disk Test



(a) web 01 with 40G disk



(b) web 02 with 40G disk

Figure 8: Configuration of two webs based on 40G disk

Table 6: Backend request distribution for different HAProxy algorithms - 40G disk

Request	Roundrobin		Leastconn		Source	
	Web-01	Web-02	Web-01	Web-02	Web-01	Web-02
1000	500	500	445	555	0	1000
2000	1000	1000	972	1028	0	2000
3000	1500	1500	1415	1585	0	3000
4000	2000	2000	1940	2060	0	4000
5000	2500	2500	2452	2548	0	5000
6000	3000	3000	2928	3072	0	6000
7000	3500	3500	3470	3530	0	7000
8000	4000	4000	3908	4092	0	8000
9000	4500	4500	4541	4459	0	9000
10000	5000	5000	5041	4959	0	10000
11000	5500	5500	5534	5466	0	11000
12000	6000	6000	6050	5950	0	12000
13000	6500	6500	6533	6464	0	13000
14000	7000	7000	7024	6976	0	14000
15000	7500	7500	7559	7441	0	15000

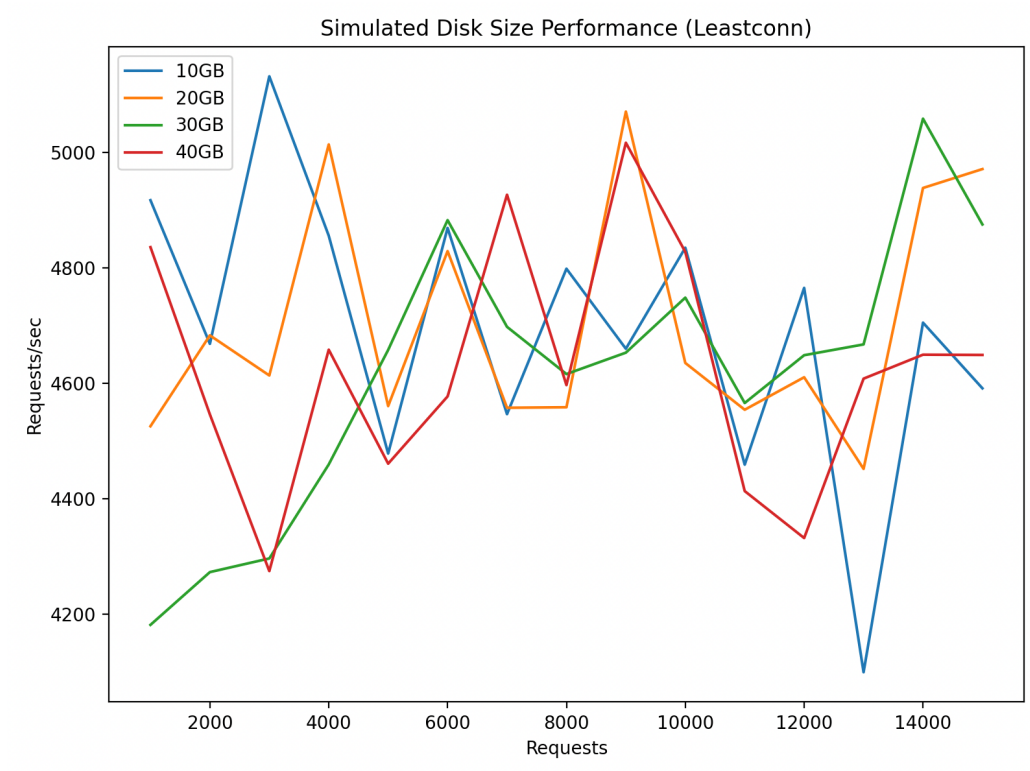


Figure 9: Leastconn Performance Under Different Disk Configs

According to the simulation experiment results above, the performance curves under

the four disk configurations (10GB, 20GB, 30GB and 40GB) all show a fluctuating trend, but overall remain in a similar range. Thus, it indicates that the disk size has a limited impact on the request processing capacity

2.3.3 CPU Test

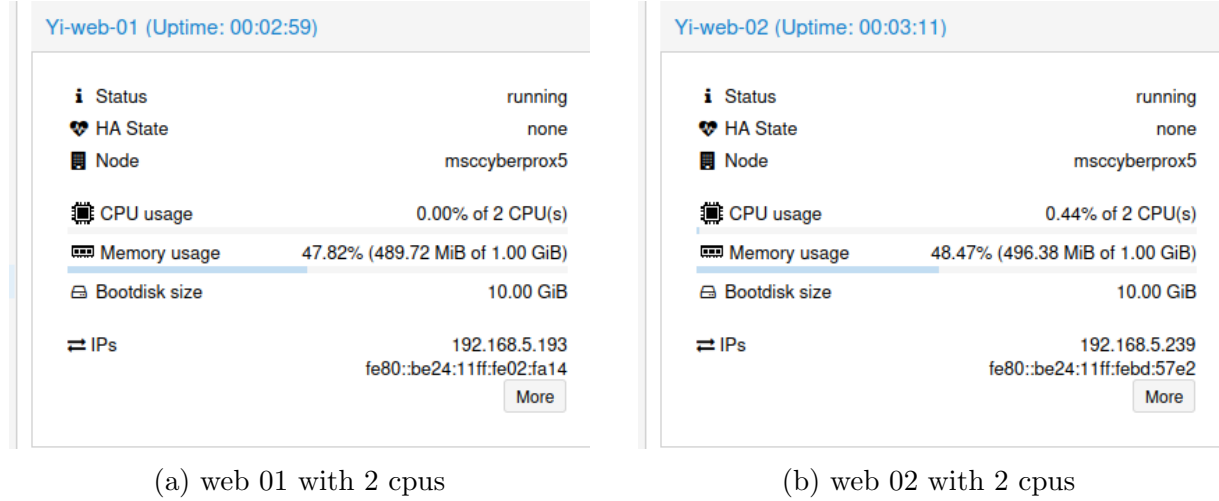


Figure 10: Configuration of two webs based on 2 cpus

Table 7: Backend request distribution for different HAProxy algorithms - 2cpus

Request	Roundrobin		Leastconn		Source	
	Web-01	Web-02	Web-01	Web-02	Web-01	Web-02
1000	500	500	491	509	0	1000
2000	1000	1000	978	1022	0	2000
3000	1500	1500	1486	1514	0	3000
4000	2000	2000	1981	2019	0	4000
5000	2500	2500	2499	2501	0	5000
6000	3000	3000	2991	3009	0	6000
7000	3500	3500	3444	3556	0	7000
8000	4000	4000	4020	3980	0	8000
9000	4500	4500	4501	4499	0	9000
10000	5000	5000	5019	4981	0	10000
11000	5500	5500	5563	5437	0	11000
12000	6000	6000	6066	5934	0	12000
13000	6500	6500	6561	6439	0	13000
14000	7000	7000	7085	6915	0	14000
15000	7500	7500	7594	7406	0	15000

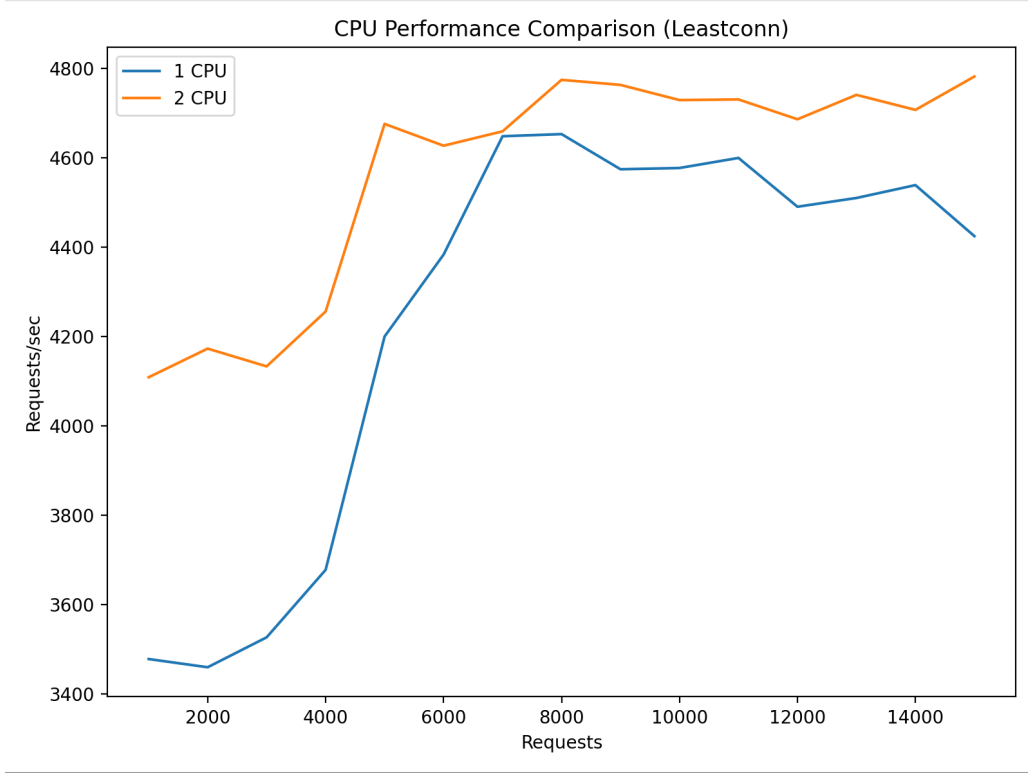


Figure 11: Leastconn Performance Under Different CPU Configs

In the CPU configuration test, the experiment compared the processing capabilities of Web services under single-core (1 CPU) and dual-core (2 cpus) conditions. As can be seen from the Figure 11, as the number of requests increases, the 2-core configuration always maintains a slightly high throughput rate. Especially when the number of requests exceeds approximately 4,000, the gap becomes more obvious. Single-core systems reach performance bottlenecks earlier and their throughput tends to stabilize, while 2-core configurations can still provide certain performance improvements under high loads. Therefore, increasing the number of CPU cores can improve concurrent processing capabilities, but the improvement is limited.

2.4 (d) VM Replication

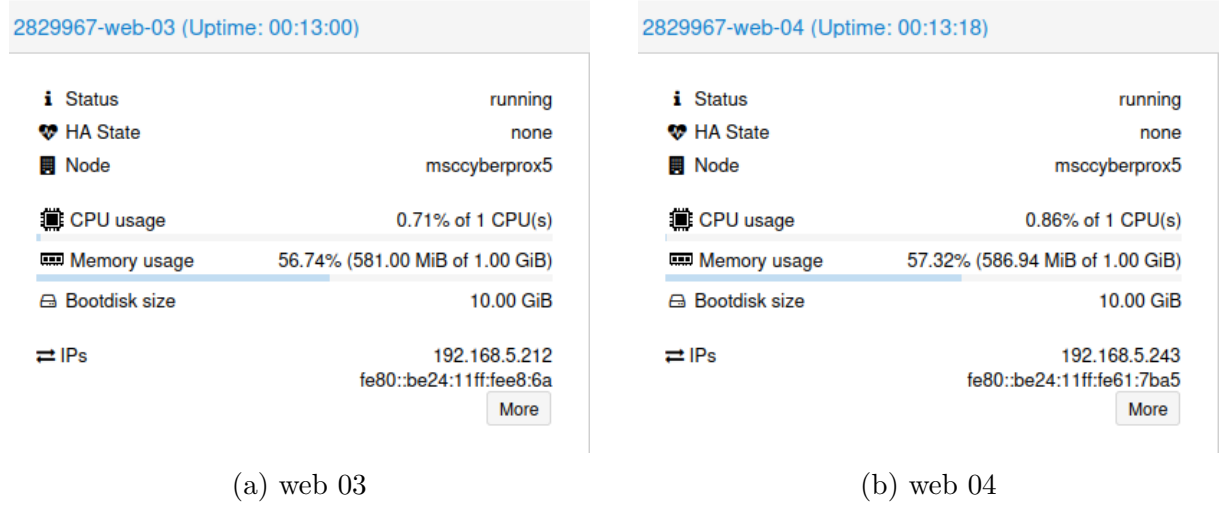


Figure 12: Configuration of two extra webs

Request	Web-01	Web-02	Web-03	Web-04
1000	253	245	251	251
2000	498	485	509	508
3000	743	734	763	760
4000	1003	982	999	1016
5000	1230	1251	1270	1249
6000	1471	1489	1526	1514
7000	1721	1739	1759	1781
8000	1997	1962	2011	2030
9000	2258	2227	2262	2253
10000	2494	2486	2501	2519
11000	2725	2743	2771	2761
12000	2969	2980	3033	3018
13000	3216	3237	3282	3265
14000	3457	3510	3523	3510
15000	3695	3776	3778	3751

In this task, only Leastconn results are shown cause other 2 algorithms have same output and the number of webs increases to 4

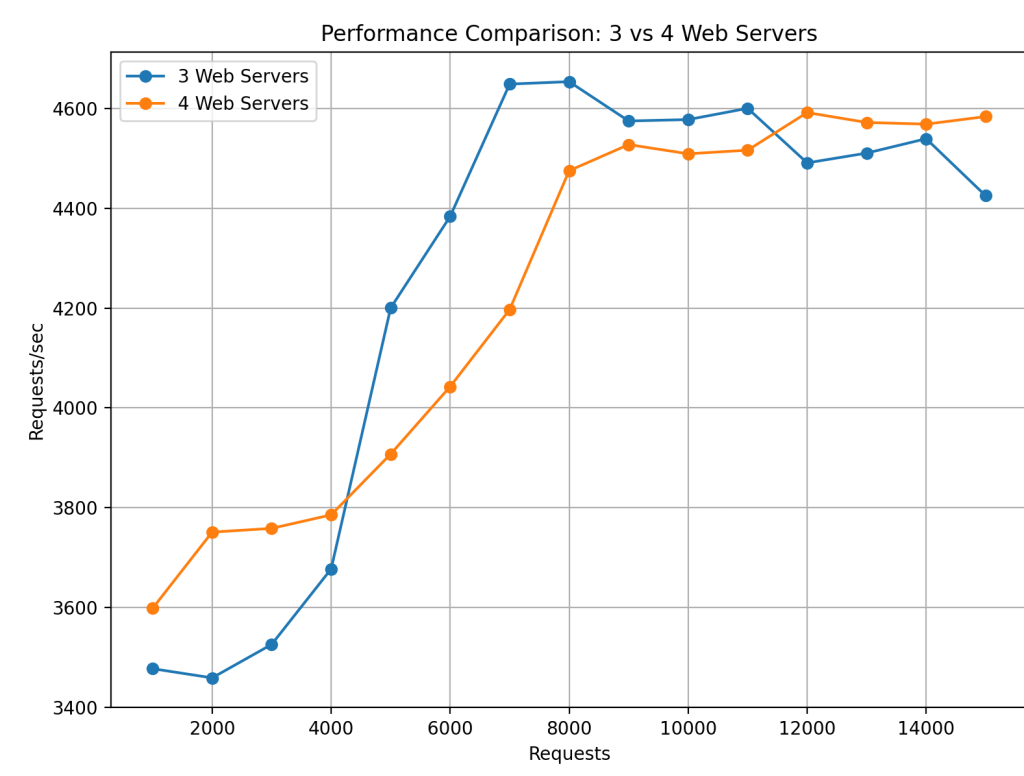


Figure 13: Leastconn Performance Under different Webs

It can be seen from the Figure 13 that when the request volume is relatively low (less than 5,000), the throughput of the three servers is slightly higher. As the number of requests increased, four servers gradually surpassed three and maintained slightly better performance under high load. However, the gap between the two is not significant, with a performance improvement of approximately only 2 to 3

3 Task 3: Access Control Matrix

3.1 (a) Access Control Matrix

Subject (Tier)	Resource (Tier)	Permitted Operations	Rationale
HAProxy	Web	FORWARD	As a load balancer, HAProxy only forwards external requests to Web and does not directly access or modify configuration.
Web	Admin	DENY	Admin is used by trusted personnel only. Blocking direct access from Web helps prevent privilege escalation after a potential Web compromise.
Web	App	READ, EXECUTE	The Web tier executes business logic (EXECUTE) via API calls to App and reads computation/output (READ), without management permission.
Admin	HAProxy	ADMIN, READ, WRITE	Admin centrally manages HAProxy: view logs (READ), update settings (WRITE), perform administrative operations (ADMIN).
App	Web	DENY	The App tier does not need to access Web, reducing attack surface.

3.2 (b) Bell-LaPadula Confidentiality

According to the assigned specification, security levels are shown as follows:

- **Level 0 (Public):** HAProxy (public entry point)
- **Level 1 (Confidential):** Web tier
- **Level 2 (Secret):** Admin tier
- **Level 3 (Top Secret):** App tier

3.2.1 (1) Simple security property (no read-up)

Under Bell-LaPadula, a subject must not read data from a higher security level. Therefore, the following flows are not allowed:

- HAProxy (0) $\rightarrow$ Web (1), Admin (2), App (3)
- Web (1) $\rightarrow$ Admin (2), App (3)
- Admin (2) $\rightarrow$ App (3)

3.2.2 (2) Star property (no write-down)

A subject must not write information to a lower security level. The following flows would violate the star property:

- App (3) → Admin (2), Web (1), HAProxy (0)
- Admin (2) → Web (1), HAProxy (0)
- Web (1) → HAProxy (0)

3.2.3 (3) Enforcement (Bell-LaPadula)

Listing 1: Excerpt of firewall rule enforcing Bell-LaPadula on web-01

```
resource "proxmox_virtual_environment_firewall_rules" "fw_web01"
{
  # Allow traffic from HAProxy to web-01
  rule { action = "ACCEPT" type = "in" proto = "tcp" source = "
    192.168.5.213" }

  # Allow only controlled requests from web-01 to App tier
  rule { action = "ACCEPT" type = "out" proto = "tcp" dest = "
    192.168.5.115" }

  # Default: block all other inbound connections
  rule { action = "DROP" type = "in" }
}
```



On	Type	Action	Macro	Interface	Protocol	Source	S.Port	Destination	D.Port	Log level	Comment
☒	in	ACCEPT			tcp	192.168.5...		192.168.5...			Allow traffic from HAProxy to web-01
☒	out	ACCEPT			tcp			192.168.5...			Allow web-01 to reach app-01
☒	in	DROP			tcp						Drop all other inbound traffic to web-01

Figure 14: Firewall rule configured for **web-01**, allowing only HAProxy inbound and App outbound traffic.



On	Type	Action	Macro	Interface	Protocol	Source	S.Port	Destination	D.Port	Log level	Comment
☒	in	ACCEPT			tcp	192.168.5...		192.168.5...			Allow traffic from HAProxy to web-02
☒	out	ACCEPT			tcp			192.168.5...			Allow web-02 to reach app-01
☒	in	DROP			tcp						Drop all other inbound traffic to web-02

Figure 15: Firewall rule configured for **web-02**, enforcing the same policy.

To enforce confidentiality according to the Bell-LaPadula model, I implemented firewall restrictions using Terraform-based Proxmox rules. For the Web tier, only inbound traffic from HAProxy is permitted, and outbound traffic is limited to the App tier. Any direct access from lower to higher levels (e.g. Web → Admin) is explicitly denied. Similarly, the Admin node blocks all inbound connections from Web servers, preventing write-down flows from higher classified tiers. This effectively enforces both “no read-up” and “no write-down” properties at network level. The example firewall configuration (Figures 14

and 15) demonstrates how direct access from lower to higher tiers is prevented, ensuring confidentiality is preserved.

3.3 (c) Biba Integrity

According to the specification, integrity levels are assigned as follows:

- **Level 3 (Highest Integrity):** Admin
- **Level 2:** App
- **Level 1:** Web
- **Level 0 (Lowest Integrity):** HAProxy

3.3.1 (1) Simple Integrity Property (no read-down)

A subject must not read data from a lower integrity level to avoid being affected by potentially corrupted input. The following flows are violations:

- Admin (3) → App (2), Web (1), HAProxy (0)
- App (2) → Web (1), HAProxy (0)
- Web (1) → HAProxy (0)

3.3.2 (2) Star (Property (no write-up))

A subject must not write to a higher integrity level to prevent lower-quality data from compromising trusted systems. The following are violations:

- HAProxy (0) → Web (1), App (2), Admin (3)
- Web (1) → App (2), Admin (3)
- App (2) → Admin (3)

3.3.3 (3) Enforcement of Biba integrity

Listing 2: Excerpt of firewall rule enforcing Biba on admin-01

```
resource "proxmox_virtual_environment_firewall_rules" "fw_admin01" {
  node_name = "msccyberprox5"
  vm_id     = 929013

  # Block inbound traffic from web-01 (lower integrity)
  rule {
    action = "DROP"
    type   = "in"
    proto  = "tcp"
    source = "192.168.5.211" # web-01
    comment = "Deny inbound traffic from web-01"
  }
}
```

```

# Block inbound traffic from web-02 (lower integrity)
rule {
    action    = "DROP"
    type      = "in"
    proto     = "tcp"
    source    = "192.168.5.214"    # web-02
    comment   = "Deny_inbound_traffic_from_web-02"
}

# Default: drop all other inbound traffic to admin-01
rule {
    action    = "DROP"
    type      = "in"
    comment   = "Drop_all_other_inbound_traffic_to_admin-01"
}
}

```

To enforce the Biba model, lower-tier systems are prevented from sending data to higher-integrity tiers. As shown in Figures 14 and 15, inbound connections from Web nodes to the Admin tier are explicitly dropped, thereby blocking write-up attempts from lower integrity levels. Furthermore, Web-to-App communication is only permitted via controlled API calls, where validation and sanitisation are applied before data is processed. This ensures that any potential corruption originating from lower-tier systems does not affect high-integrity components. And as shown firewall example above, inbound connections from lower-integrity Web nodes to the Admin tier are dropped. Thus, this effectively enforces the ‘no write-up’ principle in the Biba model.

3.4 (d) Conflict Identification and Resolution

The Bell-LaPadula model focuses on confidentiality (“no read-up”, “no write-down”), whereas the Biba model preserves data integrity (“no read-down”, “no write-up”). When both are applied to the same system, they impose opposite restrictions which cannot always be satisfied simultaneously. The following conflicts were identified:

Conflict	Explanation	Resolution Strategy
Bell requires high-tier App to be readable by Web (read-up allowed), while Biba prevents Web from reading App (no read-down)	Web must read business results from App, but Biba blocks this as App has higher integrity	Allow Web $\rightarrow$ App via controlled API with input/output validation. Use sanitisation to maintain integrity while preserving confidentiality.
Under Bell, App must not write to Web (no write-down), but under Biba, Web cannot write to App (no write-up)	Both models block App $\leftrightarrow$ Web write operations	Implement one-way communication via REST/RPC controlled interface. Web sends requests; App responds only with filtered results.
Admin requires both read/write access for monitoring, but models restrict it from interacting with lower tiers	Administrative control conflicts with both models	Use RBAC and isolated management channel separate from production traffic. Only authorised admin actions allowed via secure SSH/VPN.

Table 8: Summary of conflicts between confidentiality and integrity models and their solutions

In practice, a strict application of both models is not feasible due to the need for operational functionality. Therefore, the system implements a **trusted mediation layer (Web API)** to manage communication between tiers. All upward requests are validated before processing, while downward responses are sanitised to avoid data leakage. Administrative actions are performed through a dedicated high-integrity management channel. This combined strategy achieves a balanced trade-off between confidentiality and integrity.